Generating Noise with a VAE: The Gaussian Decoder

This note covers Section 6.5 of the swiss-roll notebook: `GaussianDecoder` / `VAEGaussianSingle`. The question it answers is — a trained VAE can generate points that land on the learned curve, but real data doesn't lie exactly on a curve, it *scatters* around one. How do you get a VAE to generate that scatter too, not just the curve itself?

1 The problem: a deterministic decoder can't reproduce noise

The ordinary VAE from Section 5 (`Decoder`) maps a latent sample z to a single output point:

$$\hat{x} = \text{decoder}(z)$$

This is a deterministic function — the same z always decodes to exactly the same $\hat{x}$. That's fine for tracing the learned curve (Section 6), but the real swiss-roll data was generated with `noise=0.5` — points scatter *around* the true curve, never landing exactly on it. A deterministic decoder has no mechanism to reproduce that scatter: every generated point necessarily lands precisely on the curve, no matter how many times you sample $z \sim \mathcal{N}(0, I)$ and decode it.

> **The fix, in one sentence:** apply the exact same trick the encoder already uses for the latent space — output the parameters of a distribution instead of a single point — but now on the *output* side of the network as well.

2 The Gaussian decoder: a distribution over outputs

Instead of one head producing $\hat{x}$ directly, `GaussianDecoder` uses a shared trunk feeding two linear heads, mirroring `VAEEncoder`'s `fc_mu`/`fc_logvar` structure exactly, just facing the opposite direction:

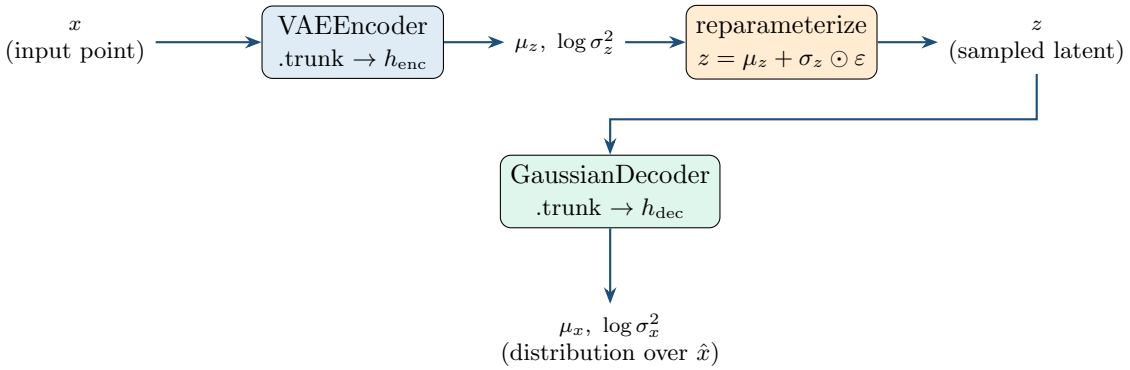
$$h = \text{trunk}(z), \qquad \mu_x = \texttt{fc_mu}(h), \qquad \log \sigma_x^2 = \texttt{fc_logvar}(h)$$

This defines a distribution over the reconstructed output, rather than a single value:

$$p_\theta(x \mid z) = \mathcal{N}\big(\mu_x(z),\, \sigma_x^2(z)\big)$$

Critically, σ_x is no longer a fixed constant (as it implicitly was for plain MSE — see Section 5 below) — it is itself a function of z, learned by the network. The decoder now predicts *where* it expects the output to be *and how confident it is* about that prediction, point by point.

3 The full architecture



Two encoder/decoder-style trunks sit back to back, one on each side of z:

- **VAEEncoder**: $x \to h_{\text{enc}} \to (\mu_z, \log\sigma_z^2)$ — compresses the input into latent-*distribution* parameters.
- **reparameterize**: turns $(\mu_z, \log\sigma_z^2)$ into one concrete sample $z = \mu_z + \sigma_z \odot \varepsilon$.
- **GaussianDecoder**: $z \to h_{\text{dec}} \to (\mu_x, \log\sigma_x^2)$ — expands the latent sample into output-*distribution* parameters.

Both halves follow the identical internal pattern — shared trunk, two linear heads — just pointed in opposite directions. `VAEEncoder` is reused completely unchanged from Section 5; only the decoder (and, as derived next, the loss) are new.

4 Deriving the loss: Gaussian negative log-likelihood

Per-dimension likelihood

Model each output dimension i as an independent Gaussian with the decoder's predicted mean and variance:

$$p_\theta(x_i \mid z) = \frac{1}{\sqrt{2\pi\sigma_{x,i}^2}} \exp\left(-\frac{(x_i - \mu_{x,i})^2}{2\sigma_{x,i}^2}\right)$$

Taking log:

$$\log p_\theta(x_i \mid z) = -\frac{(x_i - \mu_{x,i})^2}{2\sigma_{x,i}^2} - \frac{1}{2}\log\left(2\pi\sigma_{x,i}^2\right)$$

Summing over the d output dimensions (independence $\Rightarrow$ log-likelihoods add) and negating to get a loss to minimize:

$$-\log p_\theta(x \mid z) = \sum_{i=1}^{d}\left[\frac{(x_i - \mu_{x,i})^2}{2\sigma_{x,i}^2} + \frac{1}{2}\log\left(2\pi\sigma_{x,i}^2\right)\right]$$

Rewriting in `logvar` form

Substitute $\sigma^2_{x,i} = \exp(\log \sigma^2_{x,i})$ and drop the additive constant $\frac{1}{2}\log(2\pi)$ per dimension (it doesn't depend on θ, so it doesn't affect gradients or the location of the minimum):

$$-\log p_\theta(x \mid z) \;\propto\; \frac{1}{2}\sum_{i=1}^{d}\left[\log \sigma^2_{x,i} + \frac{(x_i - \mu_{x,i})^2}{\exp(\log \sigma^2_{x,i})}\right]$$

This is exactly the notebook's `recon_nll`:

$$\texttt{recon_nll} = \frac{1}{2}\sum_{i=1}^{d}\left[\texttt{logvar_x}_i + \frac{(x_i - \texttt{mu_x}_i)^2}{\exp(\texttt{logvar_x}_i)}\right]$$

Why this is a genuine generalization of MSE

Plain MSE assumes σ^2_x is a fixed constant, equal for every point (Section 5 of the earlier note, Case 2). Setting $\log \sigma^2_{x,i} = c$ for some fixed constant c collapses the expression above back to plain squared error, up to the additive constant $\frac{d}{2}c$ and multiplicative constant $\frac{1}{2e^c}$ — neither of which depends on μ_x, so gradient descent on the two objectives finds the same μ_x. **The Gaussian decoder is strictly more expressive**: it doesn't assume uniform confidence everywhere, it *learns* $\sigma_x(z)$ per point, letting the model express *"I'm confident here"* (small σ_x) versus *"there's a lot of scatter around this point in the data"* (large σ_x) — exactly the noise structure MSE has no way to represent.

The full loss

Add back the same closed-form KL term on the latent side (unchanged from Section 5 — `GaussianDecoder` only changes the reconstruction term):

$$\mathcal{L} = \underbrace{\frac{1}{2}\sum_{i=1}^{d}\left[\log \sigma^2_{x,i} + \frac{(x_i - \mu_{x,i})^2}{\sigma^2_{x,i}}\right]}_{\text{reconstruction NLL (learned variance)}} + \underbrace{\left(-\frac{1}{2}\sum_{j=1}^{k}\left(1 + \log \sigma^2_{z,j} - \mu^2_{z,j} - \sigma^2_{z,j}\right)\right)}_{\text{KL, unchanged from Section 5}}$$

5 Generating the noise: two ways to sample the output

Once trained, generation starts the same way as the plain VAE — sample z from the prior — but the Gaussian decoder offers a choice of what to do with its output:

$$z \sim \mathcal{N}(0, I) \qquad \longrightarrow \qquad \mu_x, \log \sigma^2_x = \text{GaussianDecoder}(z)$$

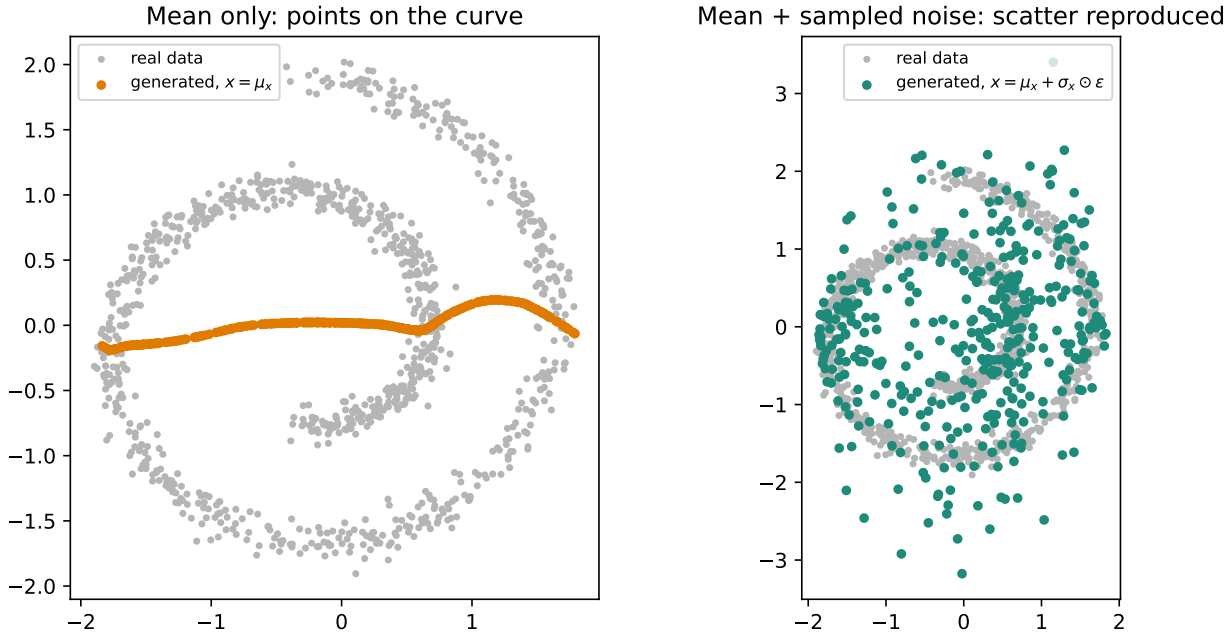
- **Mean only:** take $\hat{x} = \mu_x$ directly. This is deterministic given z, and lands exactly on the learned curve — identical in spirit to the plain VAE's generation in Section 6, just using μ_x from the new decoder instead of the old decoder's single output head.

- **Mean + sampled noise:** apply the *same reparameterization trick* on the output side that Section 5 used on the latent side:

$$\hat{x} = \mu_x + \sigma_x \odot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I)$$

This produces a genuine draw from $p_\theta(x \mid z)$ — a point scattered around the curve by an amount the decoder itself learned to predict, not landing on the curve exactly. This is what reproduces the real data's noisy scatter.

Note that `reparameterize` — originally written for the latent side in Section 5 — is literally reused unchanged here, just called on $(\mu_x, \log \sigma_x^2)$ instead of $(\mu_z, \log \sigma_z^2)$. The same trick that let gradients flow through latent sampling during training is repurposed at generation time to draw realistic output noise.



Figure, trained live from the notebook's own code (single spiral arm, Section 6.5's exact `VAEEncoder` + `GaussianDecoder` + `vae_gaussian_loss`). *Left:* generating with $\hat{x} = \mu_x$ only — every point lands on the curve the decoder learned, with no scatter, since this throws away the predicted σ_x entirely. *Right:* generating with $\hat{x} = \mu_x + \sigma_x \odot \varepsilon$ — the same underlying curve, now with realistic scatter around it, because the decoder's learned uncertainty is actually being sampled from rather than discarded.

6 Variable reference: which stage does each one belong to

With five stages now in the pipeline (input, encoder, latent sampling, decoder, generation/loss), it's easy to lose track of which variable belongs where. This table is organized by stage, in the order data actually flows through the network.

Stage 1 — Input

Variable	Meaning
x	The real data point (2D standardized swiss-roll coordinate).

Stage 2 — `VAEEncoder` (compresses x into latent-distribution parameters)

Variable	Meaning
h_{enc}	`trunk(x)` — the encoder's shared hidden representation.
μ_z	`fc_mu`(h_{enc}) — mean of $q_\phi(z \mid x)$.
$\log \sigma_z^2$	`fc_logvar`(h_{enc}) — log-variance of $q_\phi(z \mid x)$.
σ_z	$\exp(\frac{1}{2} \log \sigma_z^2)$ — *derived*, not a direct network output.

Stage 3 — `reparameterize` (turns the latent distribution into one sample)

Variable	Meaning
$\varepsilon_z \sim \mathcal{N}(0, I)$	Fresh noise, independent of the network, drawn once per forward pass.
$z = \mu_z + \sigma_z \odot \varepsilon_z$	The sampled latent code — what actually gets passed to the decoder.

Stage 4 — `GaussianDecoder` (expands z into output-distribution parameters)

Variable	Meaning
h_{dec}	`trunk(z)` — the decoder's shared hidden representation.
μ_x	`fc_mu`(h_{dec}) — mean of $p_\theta(x \mid z)$.
$\log \sigma_x^2$	`fc_logvar`(h_{dec}) — log-variance of $p_\theta(x \mid z)$.
σ_x	$\exp(\frac{1}{2} \log \sigma_x^2)$ — *derived*, not a direct network output.

Stage 5 — Generation (turns the output distribution into an actual point)

Variable	Meaning
$\varepsilon_x \sim \mathcal{N}(0, I)$	Fresh noise for the *output* side — only used at generation time, via the same `reparameterize` function reused on $(\mu_x, \log \sigma_x^2)$.
$\hat{x} = \mu_x$	Mean-only generation — deterministic, lands on the curve.
$\hat{x} = \mu_x + \sigma_x \odot \varepsilon_x$	Mean+noise generation — a genuine draw from $p_\theta(x \mid z)$.

Stage 6 — Loss (combines variables from multiple stages)

Variable	Built from	Meaning
`recon_nll`	x, μ_x, $\log \sigma_x^2$	Gaussian NLL — *output*-side term (Section 4).
`kl_loss`	μ_z, $\log \sigma_z^2$	Closed-form KL — *latent*-side term, unchanged from Section 5.
`total_loss`	both of the above	`recon_nll + kl_loss`, the value `.backward()` is called on.

The pattern worth noticing: z is the only variable that appears in *both* halves of the network — it's simultaneously the encoder's output and the decoder's input, the single point where the two trunks connect. Every other variable belongs cleanly to one stage: x-side variables

$(\mu_z, \sigma_z, \varepsilon_z)$ never appear in the decoder's math, and $\hat{x}$-side variables $(\mu_x, \sigma_x, \varepsilon_x)$ never appear in the encoder's math. The subscripts z and x throughout this note exist precisely to keep these two symmetric but separate halves from being confused with each other.

Summary

1. A plain VAE decoder is deterministic — one z, one $\hat{x}$ — so it can trace a learned curve but can never reproduce the scatter real noisy data has around that curve.

2. The fix mirrors the encoder's own trick, applied to the output: replace a single output head with two heads, μ_x and $\log \sigma_x^2$, defining $p_\theta(x \mid z) = \mathcal{N}(\mu_x, \sigma_x^2)$ instead of a point.

3. The reconstruction loss becomes the Gaussian negative log-likelihood, derived directly from that assumed distribution — and reduces exactly to plain MSE as the special case of fixed, un-learned variance.

4. Generation now offers a genuine choice: decode μ_x alone for points exactly on the curve, or reparameterize $\mu_x + \sigma_x \varepsilon$ (the same trick used for the latent code, now reused on the output) for points that reproduce the data's actual noisy scatter.